

AI powered speech-to-speech mock interview platform

Noel Biju, Adwaith V, Faiha Safeer, Aisha Mol AA

Dept. of Computer Science & Engineering, College of Engineering Poonjar, Kottayam, Kerala, India

noelbijuk@gmail.com | unnikuttan8t@gmail.com | faihasafeer43@gmail.com | aishamolaa2@gmail.com

1. Introduction

1.1 Purpose

This System Architecture Document (SAD) describes the overall design of the AI-powered speech-to-speech mock interview platform. It defines the system's structure, its major components, how those components interact, and the end-to-end workflow of a single interview session. It is intended as a technical reference for development, guiding how new modules should be built and where they fit within the larger system, and as supporting documentation for the project's academic evaluation.

1.2 Scope

This document covers the architecture of the minimum viable product (MVP) as defined in the project's requirement analysis: a single-session, turn-based, resume - personalized voice interview system. It notes, where relevant, how the architecture is designed to extend toward the production-grade version described separately, without requiring a redesign.

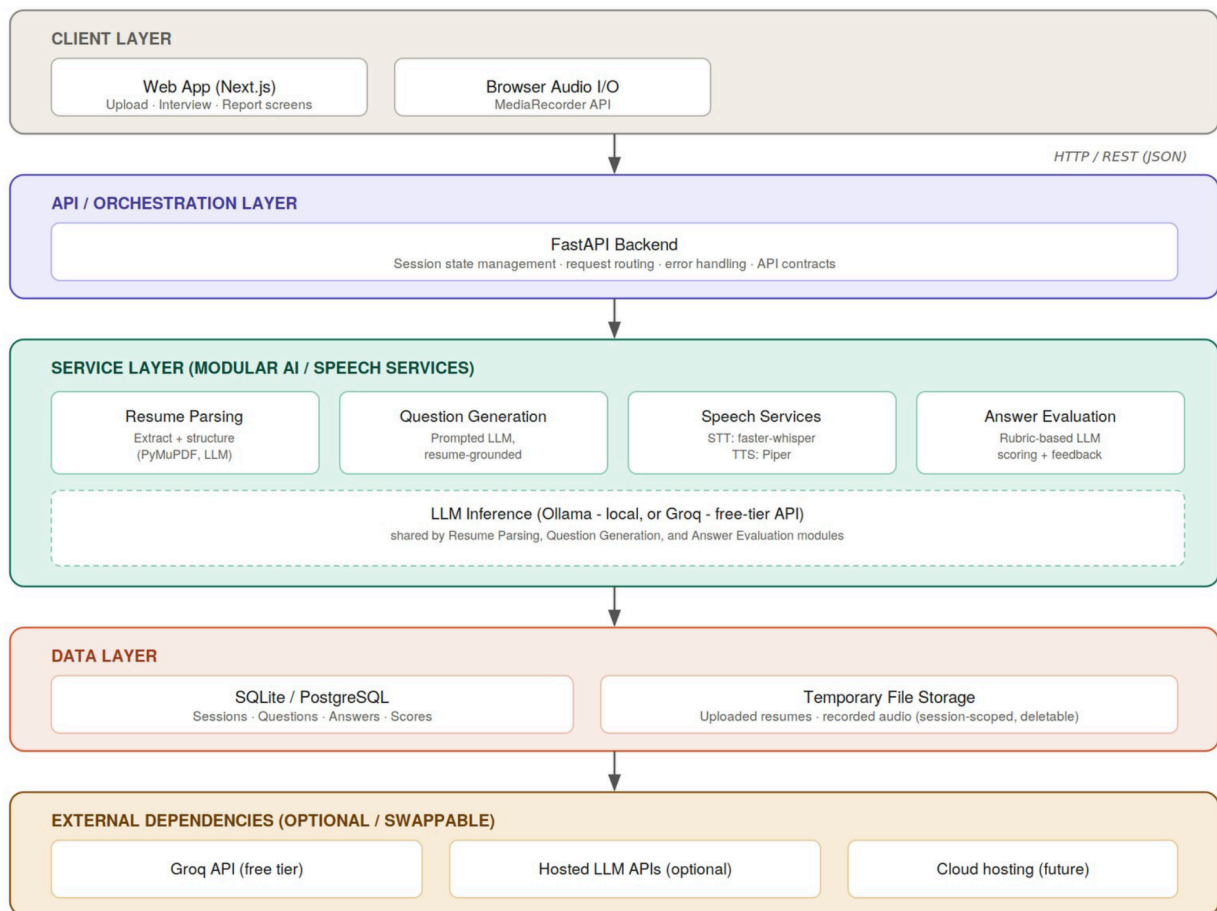
1.3 Architectural Goals

- Modularity - each functional concern (speech, AI reasoning, orchestration, presentation) is isolated into its own component so it can be built, tested, and replaced independently.
- Low cost — every component in the MVP architecture has a free or open-source implementation option.
- Swappability — local components (Ollama, faster-whisper, Piper) and hosted equivalents (Groq, cloud STT/TTS APIs) are interchangeable behind the same interface, supporting the documented migration path to a production-scale version.
- Reliability - the orchestration layer validates and handles failure at each step rather than allowing a single failed call to break an entire session.

2. System Overview

The system is structured as a layered, modular architecture with five layers: a client layer (the web application the candidate interacts with), an API/orchestration layer (the central coordinator), a service layer (the individual AI and speech modules), a data layer (persistent storage), and a set of optional external dependencies that can be swapped in as the system scales. Each layer communicates with the layer directly above and below it through a defined interface (primarily HTTP/REST with JSON payloads), and no layer reaches past its immediate neighbour the client layer, for example, never calls the service layer directly; it always goes through the orchestration layer.

3. Architecture Diagram



4. Major Components

4.1 Client Layer - Web Application

Built with Next.js, this is the only layer the candidate directly interacts with. It handles resume upload, renders the turn-based interview screen, captures audio via the browser's MediaRecorder API, and displays the final feedback report. It holds no business logic - every decision (what question comes next, how an answer is scored) is delegated to the backend, keeping the frontend simple and replaceable.

4.2 API/ Orchestration layer - FastAPI Backend

This is the architectural backbone described in the project's backend skeleton. It exposes REST endpoints, manages session state (which question a candidate is on, what's been answered so far), and coordinates calls to the service layer in the correct order. It is the only component that talks to the database directly, and the only component the client layer talks to every other module is reached

indirectly through this layer, which is what allows individual service modules to be modified or replaced without touching the frontend.

4.3 Service layer - Modular AI and Speech services

Four independent modules live in this layer, each callable by the orchestration layer as a discrete function or service:

- Resume Parsing extracts raw text from an uploaded PDF/DOCX and structures it into JSON (skills, projects, experience, education) using an LLM.
- Question Generation takes the structured resume data and produces a personalized set of interview questions via prompted LLM calls.
- Speech Services handles both directions of audio: speech-to-text (faster-whisper) for transcribing candidate answers, and text-to-speech (Piper) for voicing questions.
- Answer Evaluation scores each transcribed answer against a defined rubric and produces short qualitative feedback, again via a prompted LLM call.

All three LLM-dependent modules (Resume Parsing, Question Generation, Answer Evaluation) share a common LLM inference component, which can point to a local Ollama instance or a hosted API such as Groq without changing the calling code in any of the three modules this is the core swappability mechanism referenced in the project's economic and production-scale documentation.

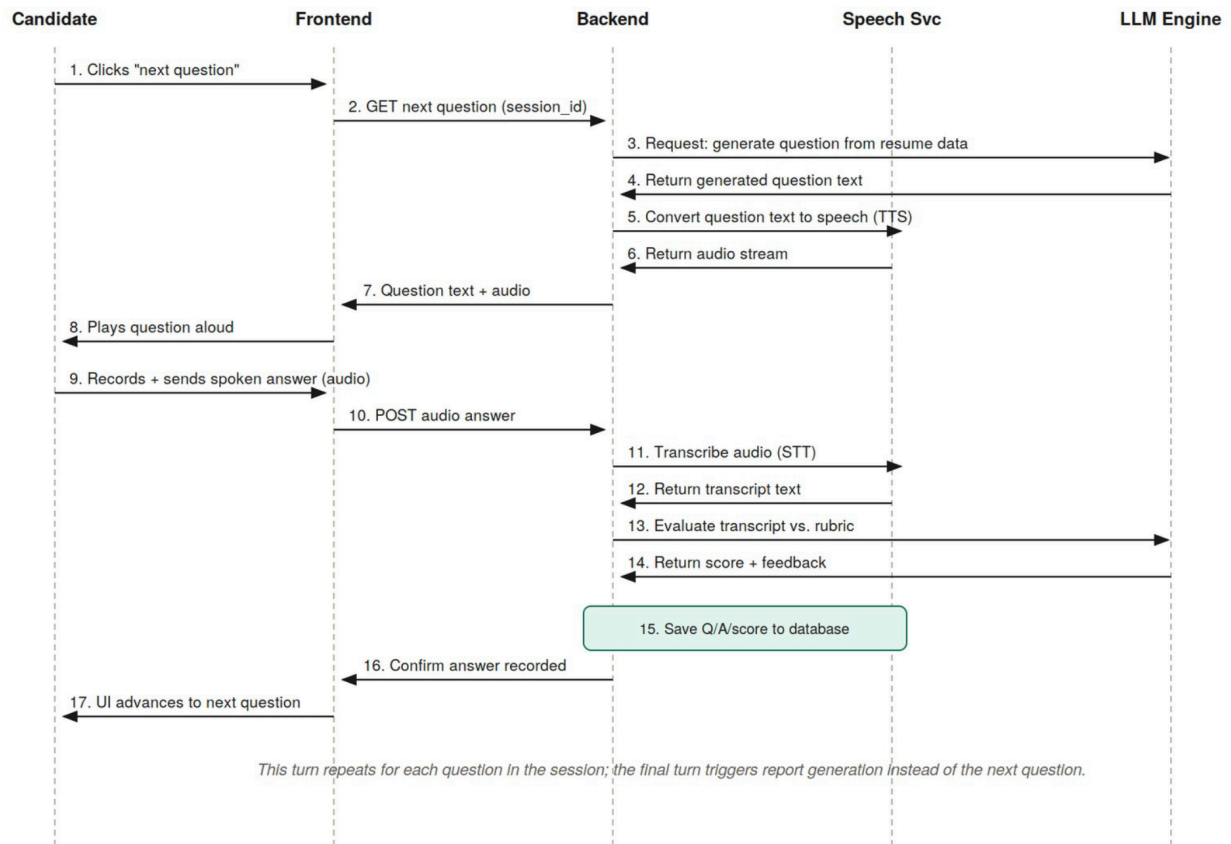
4.4 Data Layer

Two persistence mechanisms: a relational database (SQLite for the MVP, swappable to PostgreSQL for production) storing sessions, questions, answers, and scores; and temporary file storage for uploaded resumes and recorded audio, which is session-scoped and deletable, consistent with the data-handling practices defined in the project's feasibility study.

4.5 External Dependencies

These are not required for the MVP to function but are architected as drop-in replacements for their local equivalents: the Groq API as a faster alternative to local LLM inference, other hosted LLM APIs (OpenAI, Anthropic) as a demo-day reliability fallback, and cloud hosting infrastructure for a future public deployment.

5. Component Interaction - Sequence of a single Interview turn



6. Conclusions

This architecture organizes the system into five clearly bounded layers, each communicating through defined interfaces rather than direct dependencies on implementation details. This structure directly supports the project's core constraints - building at low cost, on limited hardware, within a tight timeline — while leaving a documented, low-friction path to a production-scale version as the system's needs grow.